

Приложение Телефонная книга.

Тема:

разработка приложения под Windows в технологии ООП.

Цель:

Объектно-ориентированный анализ, проектирование и реализация приложения под Windows «Телефонная книга».

В процессе выполнения работы студенты изучают:

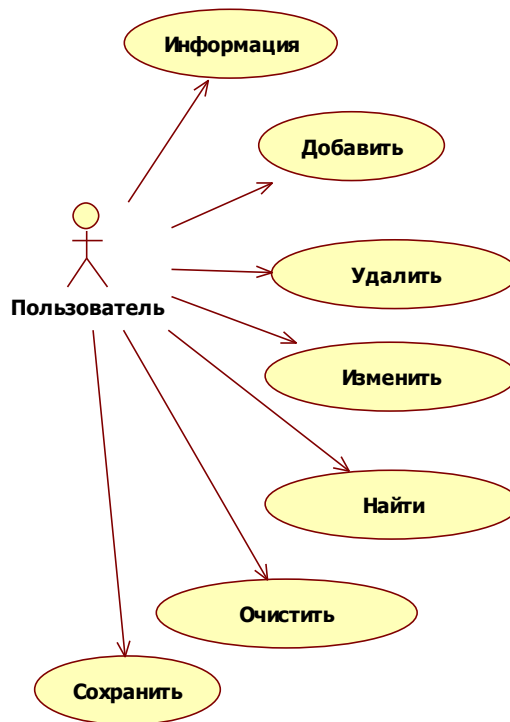
- отношения между классами: ассоциация, агрегация, зависимость, их реализацию средствами языка программирования высокого уровня;
- этапы разработки приложений в технологии ООП;
- элементы технологии визуального программирования;
- потоковые классы для работы с файлами на диске;
- диаграммы языка UML для документирования разработки.

Функциональные требования к приложению.

Приложение должно обеспечивать пользователю:

- ввод, редактирование и сохранение имён абонентов городской телефонной сети и номеров их телефонов
- записи должны храниться и отображаться в отсортированном по именам порядке;
- поиск по имени;
- удаление записи;
- очистку книги.

Функциональные требования представлены диаграммой прецедентов (use-case диаграммой) расположенной ниже.



Сценарий для прецедента «Добавить»

Основной поток событий

- 1) Пользователь вводит имя абонента.
- 2) Пользователь вводит номер абонента.
- 3) Пользователь нажимает команду Добавить.
- 4) Приложение добавляет абонента в книгу и отображает на экране запись по нему в списке.

Альтернативный поток событий 1. Одно из полей записи не заполнено.

- 3.1. Пользователь получает окно с сообщением.
- 3.2. Приложение переходит в режим Ввод и редактирования незаполненного поля.

Диаграмма классов модели объектно-ориентированного анализа.

Проанализировав прецеденты можно выделить следующие классы для реализации функциональности приложения. Они представлены на диаграмме классов ниже.

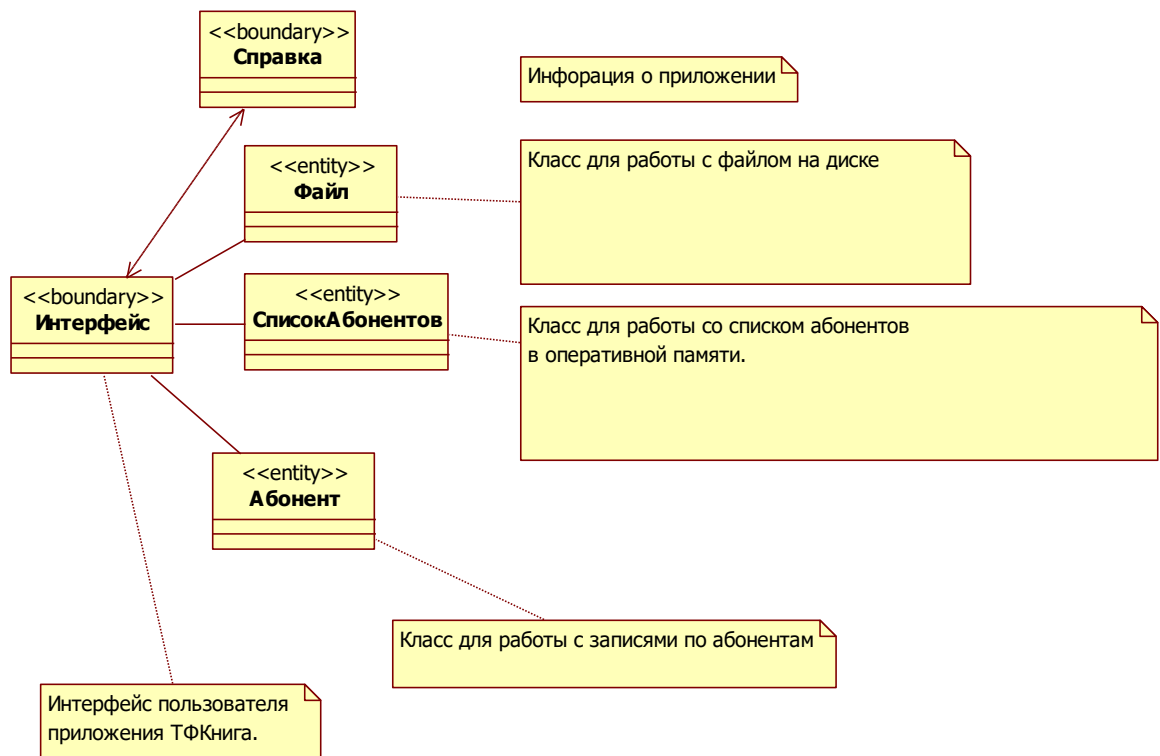
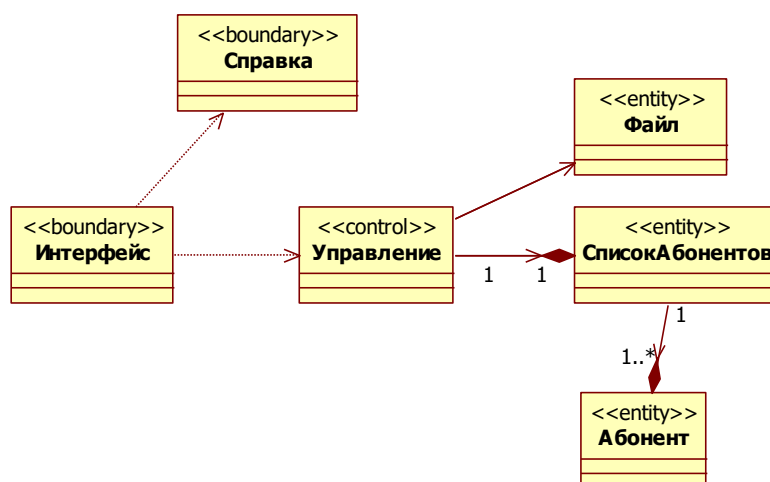


Диаграмма классов проекта

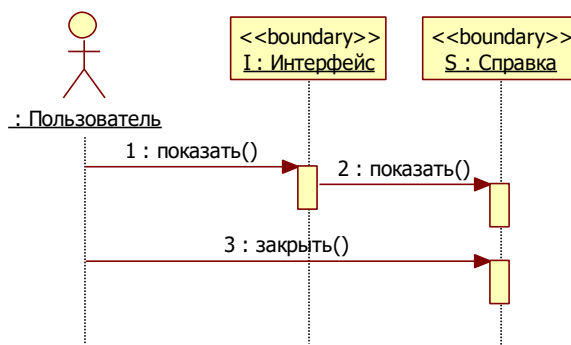
Проанализировав сообщения, которыми обмениваются классы в процессе выполнения прецедентов можно построить следующую диаграмму классов проекта.

Для упрощения взаимодействия между классами в процессе работы приложения добавим класс «Управление». Тогда наша диаграмма примет следующий вид:

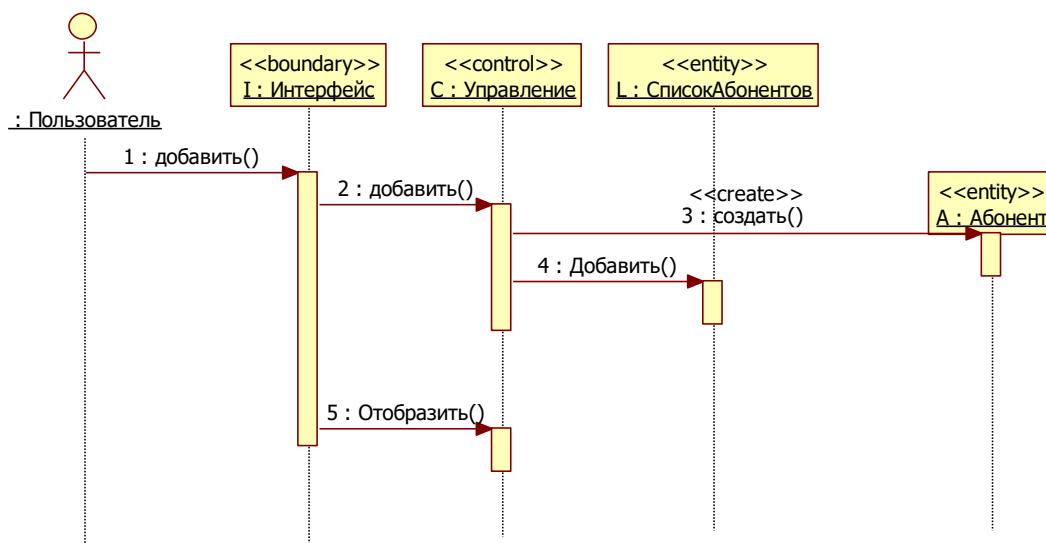


Из диаграммы классов видно, что объект класса «Интерфейс» вызывает методы класса «Управление» и «Справка». Объект же класса «Управление» в свою очередь вызывает методы объектов классов «Файл», «Список абонентов», «Абонент».

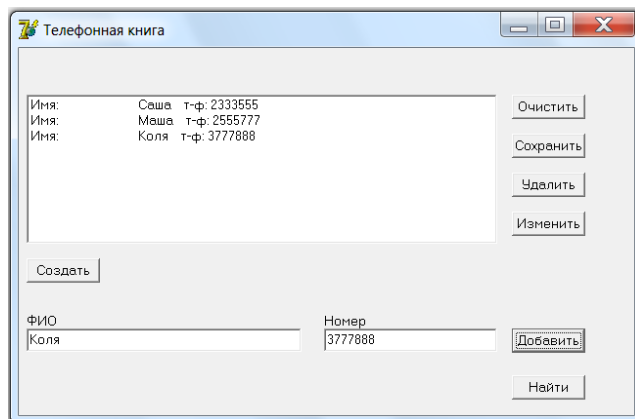
Обмен сообщениями между объектами. Диаграмма последовательностей.
Давайте спроектируем обмен сообщениями между объектами в процессе выполнения прецедента «Информация». На диаграмме последовательностей приведённой ниже приведёна последовательность сообщений между объектами в процессе реализации прецедента «Информация».



На диаграмме последовательностей приведённой ниже приведёна последовательность сообщений между объектами в процессе реализации прецедента «Добавить».



Интерфейс приложения может выглядеть так:



Порядок выполнения разработки

- Реализуйте по заданной спецификации класс «Абонент». и протестируйте его в режиме консольного приложения. Для этого создайте новое консольное приложение и добавьте в него модуль и поименуйте его UAbonent.
- Реализуйте по заданной спецификации класс «Список абонентов». и протестируйте его в режиме консольного приложения. Для этого создайте новое консольное приложение и добавьте в него модуль и поименуйте его UAbonentList.
- Создайте новый проект приложение под Windows. Сохраните его под именем RTFBook. Переименуйте форму именем Panel. Тогда имя класса формы будет автоматически заменено на TPanel. Сохраните модуль формы под именем UPanel. В этом модуле реализуйте по заданной спецификации класс «Интерфейс» и протестируйте его.
- Вставьте в проект модуль и поименуйте его UAbout с библиотечным окном TAboutBox. В этом модуле реализуйте класс «Справка» и протестируйте его вместе с приложением. Для вызова окна «Справка» используйте меню Справка Интерфейса приложения.
- Вставьте в проект модуль и поименуйте его UControl. В этом модуле реализуйте по заданной спецификации класс «Управление» и протестируйте его вместе с классом «Интерфейс».

- Вставьте в проект модуль и поименуйте его UControl. В этом модуле реализуйте по заданной спецификации класс «Управление» и протестируйте его вместе с классом «Интерфейс».
- Вставьте в проект модули UAbonentList («Список абонентов») и UAbonent («Абонент») протестируйте приложение «Телефонная книга».